

Лабораторная работа № 5

Тема: Подключение Prisma ORM к существующей базе shop_db

Цель работы: Подключить Prisma ORM к существующему Express + PostgreSQL проекту, получить Prisma-модели из уже созданной базы и выполнить первые операции чтения через Prisma Client.

В этой лабораторной работе **не создаём новую базу и не удаляем существующие таблицы.**

Используем:

- shop_db

с таблицами:

- users
- user_profiles
- orders
- order_items
- products
- categories

Полный CRUD через Prisma будет темой следующей лабораторной работы.

1. Проверьте Node.js

```
node --version
```

Для используемого в курсе Prisma ORM 7 подходит Node.js 22.x.

2. Проверьте package.json

Проект должен использовать ES Modules:

```
{  
  "type": "module"  
}
```

3. Установите Prisma ORM

В каталоге express-shop:

```
npm install -D prisma@7 typescript tsx @types/node @types/pg
```

```
npm install @prisma/client@7 @prisma/adapters pg dotenv
```

Prisma ORM 7 требует PostgreSQL driver adapter для текущего Prisma Client; официальная документация для PostgreSQL использует @prisma/adapters вместе с pg.

4. Инициализируйте Prisma

```
npx prisma@7 init --datasource-provider postgresql --output
```

```
../generated/prisma
```

После выполнения должны появиться Prisma-файлы проекта.

5. Проверьте структуру

Пример:

```
express-shop/
├── prisma/
│   └── schema.prisma
├── generated/
├── config/
├── controllers/
├── routes/
├── services/
├── prisma7.config.ts
├── .env
├── app.js
└── package.json
```

6. Настройте .env

Добавьте:

```
DATABASE_URL="postgresql://shop_user:college123@localhost:5432/shop_db?schema=public"
```

Используйте реальные параметры вашей базы, если они отличаются.

7. Не удаляйте старые переменные сразу

Если проект пока использует:

PGHOST

PGPORT

PGDATABASE

PGUSER

PGPASSWORD

можно временно оставить их.

Во время перехода будут существовать два способа подключения:

pg Pool

Prisma

8. Проверьте Prisma Config

Файл должен указывать существующий schema.prisma и DATABASE_URL.

Пример:

```
import "dotenv/config";
```

```
import {
  defineConfig
} from "prisma/config";
```

```
export default defineConfig({
  schema: "prisma/schema.prisma",
```

```
  migrations: {
```

```

    path: "prisma/migrations"
  },

  datasource: {
    url: process.env.DATABASE_URL
  }
});

```

9. Получите Prisma-модель существующей базы

Выполните:

```
npx prisma@7 db pull
```

db pull выполняет introspection: подключается к существующей БД, анализирует её схему и преобразует таблицы, столбцы, ключи и отношения в Prisma-модели.

10. Откройте schema.prisma

После db pull найдите модели, соответствующие:

products

categories

users

orders

order_items

user_profiles

Не изменяйте базу данных.

Сначала изучите полученный результат.

11. Найдите модель товара

Она будет отражать существующую таблицу products.

Обратите внимание на:

- id;
- name;
- price;
- category_id;
- in_stock;
- связь с categories;
- связь с order_items.
-

12. Приведите имя модели к удобному виду

Для учебного проекта удобно использовать:

```

model Product {
  id      Int    @id @default(autoincrement())
  name    String @db.VarChar(150)
  price   Int
  categoryId Int  @map("category_id")
}

```

```
inStock Boolean @default(true) @map("in_stock")
```

```
    @@map("products")  
}
```

Здесь физическая таблица остаётся:

products

но в JavaScript можно использовать:

Product

categoryId

inStock

13. Настройте Category

```
model Category {
```

```
  id Int @id @default(autoincrement())
```

```
  name String @unique @db.VarChar(100)
```

```
  products Product[]
```

```
    @@map("categories")  
}
```

14. Добавьте связь Product → Category

Модель Product должна содержать:

```
category Category @relation(  
  fields: [categoryId],  
  references: [id]  
)
```

Полный фрагмент:

```
model Product {
```

```
  id Int @id @default(autoincrement())
```

```
  name String @db.VarChar(150)
```

```
  price Int
```

```
  categoryId Int @map("category_id")
```

```
  inStock Boolean @default(true) @map("in_stock")
```

```
  category Category @relation(  
    fields: [categoryId],  
    references: [id]  
  )
```

```
    @@map("products")  
}
```

Существующий FOREIGN KEY PostgreSQL при этом не исчезает.

15. Сгенерируйте Prisma Client

`npx prisma@7 generate`

Prisma Client генерируется на основании `schema.prisma`; после изменения моделей клиент нужно генерировать повторно.

16. Проверьте generated

После генерации должен появиться каталог:

```
generated/  
└── prisma/
```

Внутри находится сгенерированный Prisma Client.

Не редактируйте эти файлы вручную.

17. Создайте config/prisma.js

```
import "dotenv/config";
```

```
import {  
  PrismaPg  
} from "@prisma/adapter-pg";
```

```
import {  
  PrismaClient  
} from "../generated/prisma/client";
```

```
const adapter =  
  new PrismaPg({  
    connectionString:  
      process.env.DATABASE_URL  
  });
```

```
export const prisma =  
  new PrismaClient({  
    adapter  
  });
```

Для долгоживущего серверного приложения рекомендуется создавать один экземпляр `PrismaClient` и повторно использовать его во всех запросах.

18. Измените команду запуска

Поскольку современный генератор Prisma создаёт TypeScript-код клиента, для учебного проекта используем `tsx`.

В `package.json`:

```
{  
  "scripts": {  
    "start": "tsx app.js"
```

```
}  
}
```

19. Создайте тест Prisma

Создайте:

test-prisma.js

```
import {  
  prisma  
} from "../config/prisma.js";
```

```
const products =  
  await prisma.product.findMany();
```

```
console.log(products);
```

20. Запустите

`npx tsx test-prisma.js`

Ожидается массив товаров из существующей таблицы PostgreSQL.

21. Добавьте сортировку

```
const products =  
  await prisma.product.findMany({  
    orderBy: {  
      id: "asc"  
    }  
  });
```

22. Выполните поиск по id

```
const product =  
  await prisma.product.findUnique({  
    where: {  
      id: 1  
    }  
  });
```

```
console.log(product);
```

Используйте существующий id.

23. Проверьте отсутствующий id

```
const product =  
  await prisma.product.findUnique({  
    where: {  
      id: 999999  
    }  
  });
```

```
});
```

```
console.log(product);
```

Ожидается:

null

findUnique() предназначен для поиска одной записи по уникальному полю, включая primary key.

24. Проверьте фильтрацию

```
const products =  
  await prisma.product.findMany({  
    where: {  
      price: {  
        gt: 10000  
      },  
      inStock: true  
    }  
  });
```

```
console.log(products);
```

25. Получите категорию вместе с товаром

```
const products =  
  await prisma.product.findMany({  
    include: {  
      category: true  
    },  
  
    orderBy: {  
      id: "asc"  
    }  
  });
```

```
console.log(products);
```

Это только первое знакомство со связанными объектами.

Подробно include, select и запросы отношений будут рассмотрены в отдельной лекции.

26. Удалите тестовый файл

После проверки:

test-prisma.js

можно удалить.

27. Перепишите getAllProducts()

Откройте:

services/productService.js

Импортируйте Prisma:

```
import {
  prisma
} from "../config/prisma.js";
```

Реализуйте:

```
export async function getAllProducts() {
  return prisma.product.findMany({
    include: {
      category: true
    },

    orderBy: {
      id: "asc"
    }
  });
}
```

28. Перепишите findProductById()

```
export async function findProductById(
  id
) {
  return prisma.product.findUnique({
    where: {
      id
    },

    include: {
      category: true
    }
  });
}
```

29. Controller менять не требуется

Пример:

```
export async function getProductById(
  req,
  res,
  next
) {
  try {
    const id =
      Number(req.params.id);
```

```

const product =
  await findProductById(id);

if (!product) {
  return next(
    new AppError(
      "Product not found",
      404
    )
  );
}

res.json(product);
} catch (error) {
  next(error);
}
}

```

30. Запустите Express

npm start

31. Проверьте получение списка

GET /api/products

Данные должны поступать из той же PostgreSQL-базы, но теперь Service использует Prisma Client.

32. Проверьте получение товара

GET /api/products/1

Используйте существующий id.

33. Проверьте 404

GET /api/products/999999

Ожидается:

404 Not Found

Prisma возвращает:

null

а Controller преобразует это в HTTP 404.

34. Запустите Prisma Studio

npx prisma@7 studio

Откройте адрес, который покажет CLI.

Посмотрите:

Product

Category

User

Order

OrderItem

если эти модели присутствуют в схеме.

35. Измените данные через PostgreSQL

Например, через pgAdmin измените цену товара.

После этого снова выполните:

GET /api/products

Prisma должен вернуть новое значение.

Это подтверждает, что Prisma работает с той же PostgreSQL-базой, а не с отдельным хранилищем.

36. Сравните старый и новый код

Было:

```
const result =  
  await pool.query(  
    `SELECT *  
      FROM products  
      WHERE id = $1`,  
    [id]  
  );
```

return result.rows[0];

Стало:

```
return prisma.product.findUnique({  
  where: {  
    id  
  }  
});
```

37. Не удаляйте pg-версию сразу

На этапе изучения полезно сохранить копию старого Service.

Например:

productService.sql.js

productService.js

Это позволит студенту сравнить:

SQL

↓

Prisma

и понимать, какую операцию ORM выполняет вместо ручного запроса.

Итоговая структура

```
express-shop/
├── config/
│   └── prisma.js
├── controllers/
│   └── productController.js
├── generated/
│   └── prisma/
├── prisma/
│   └── schema.prisma
├── routes/
│   └── productRoutes.js
├── services/
│   └── productService.js
├── prisma7.config.ts
├── .env
├── app.js
└── package.json
```

Требования к результату

Необходимо:

- использовать существующую shop_db;
- установить Prisma ORM;
- настроить DATABASE_URL;
- выполнить prisma db pull;
- получить модели существующих таблиц;
- использовать @map и @@map там, где это необходимо;
- выполнить prisma generate;
- создать один экземпляр PrismaClient;
- использовать PostgreSQL adapter;
- получить товары через findMany();
- получить один товар через findUnique();
- подключить Prisma к Product Service;
- сохранить существующий Controller;
- сохранить существующие REST endpoints;
- открыть данные через Prisma Studio;
- не удалять существующую PostgreSQL-базу.